



An AI agent that does real Salesforce work – and **structurally cannot** write to production on its own.

Coding agents can already reach a Salesforce org. What they lack is the method a good admin has: describe before you reference, dry-run before you deploy, verify in the org rather than trusting the response, capture before-values so a bulk change can be undone, and never touch production by accident.

Torque is that method, packaged — skills, rules, a validation harness, and enforcement that binds. It deploys and proves the deploy landed, updates in bulk with an undo trail, audits read-only, verifies in a real browser, and logs every change for handoff. Production writes need a token an operator mints at a terminal; the agent can ask for one and provably cannot issue one.

SPECIFICATION			
Org credentials stored	none — sf CLI only	Licence	MIT · open source
Surface	Claude Code workspace	Org path	sf CLI + official MCP
Skills · subagents	5 · 2	Enforcement	2 hooks, 4 tool surfaces
Validation	27 checks, live org	Adversarial fixtures	128
Self-proof	11 mutators	Try it with no org	3 seconds

01	What it actually does	capabilities
02	Why this isn't the MCP server	positioning
03	Setup — clone to working	10 minutes
04	The operations, worked through	the manual
05	The limit — how production is gated	safety model
06	When it refuses	troubleshooting
07	Proving it — the validation harness	evidence
08	Where it came from	provenance
09	Reference	commands · files

Omid Mojtahedi · Salesforce implementation lead · github.com/omoji-personal/torque

What it actually does

Asking an agent to "add a field" should mean the field exists, the right profiles can see it, and there is a record of the change — not that one API returned 200. Each skill encodes the whole sequence, including the parts people skip when they're tired.

DEPLOY, THEN PROVE IT

Dry-run → deploy → SOQL-verify the component exists → verify field-level security → confirm zero residue on teardown. Formula fields get no automatic FLS from the Metadata API; the pipeline catches that instead of shipping an invisible field.

MASS-UPDATE WITH AN UNDO

Query current values, diff against intent, show counts, touch only records that actually change. Before-values are captured first, so the operation is reversible — and an empty diff stops rather than issuing a no-op write.

AUDIT AN ORG, READ-ONLY

A fixed survey — objects, active flows and triggers, validation rules, permission assignments, setup audit trail — dispatched to a subagent that carries no Edit or Write tools, so a large org doesn't flood the main context.

RUN APEX UNDER REVIEW

Inline Apex is refused outright. You read the exact body at a terminal, approve it, and the gate re-hashes the file at run time — so a payload edited after approval fails on digest mismatch.

VERIFY IN THE BROWSER

Metadata deploying is not the page rendering. A real headless Chromium follows the frontdoor redirect and asserts the Lightning shell came up — over a session token that never touches stdout.

KEEP A HANDOFF RECORD

Per-org session log: what changed, before → after, records touched by Id, what was verified, what still needs human UAT. Redacted — no PII, no raw rows, only what undo and the next session need.

THE LIMIT

All of that runs freely against a sandbox or developer org. **Production is different: the agent cannot write there on its own.** Not "asks first" — structurally gated. A production write needs a single-use token minted by an operator at a real terminal, or a bounded, revocable session window. The agent can request one; it provably cannot mint one.

Who it's for

Consultancies and admins running AI-assisted work in orgs they don't own, where the cost of a wrong write is somebody else's production data — and anyone who wants to use an agent on a real org without watching every keystroke. Torque adds no org access of its own: it runs on the Salesforce CLI and the official MCP server, and stores no org credentials.

Why this isn't the MCP server

The official Salesforce MCP server and the `sf` CLI are good, and Torque runs on top of both. But they are a **tool surface**: they give an agent hands. They take no position on whether an operation is wise, whether it worked, or whether the org on the other end is a client's production instance. Filling that gap is the whole job.

	SALESFORCE MCP + SF CLI	GEARSET / COPADO	SFDX-HARDIS + CI LIBRARIES	TORQUE
Exposes org operations to an agent	yes	—	human / CI	builds on
Encodes <i>how</i> to do the work	—	UI-guided	for humans	for agents
Verifies the outcome in the org	—	partial	partial	yes
Distinguishes prod at write time	trusts alias	env model	trusts alias	live query
Proves its own checks can fail	—	—	—	11 mutators
Rollback across environments	—	yes	—	single-org only
Works outside Claude Code	yes	yes	yes	no
Vendor support / SLA	yes	yes	community	none

Torque loses the last three rows on purpose. It is a workspace layer, not a release-management platform: if you need environment-to-environment rollback or a vendor to call at 2am, buy the platform. What it does that they don't is govern an *agent* operating an org in real time.

Three things I could not find anywhere else

1 — It treats "green but wrong" as the real failure mode. The expensive incidents are rarely errors; they are successes that were false. A deploy returns success and the field is invisible because formula fields receive no FLS. A flow reads Active in one API and is absent in another. An agent reports a task complete because the tool said so. Every Torque operation ends by asking the *org* what is true, not by reading its own return code.

2 — It never trusts the alias. Every other column decides what org it is talking to by believing the string you passed. `--target-org acme-sandbox` can point anywhere, and the most likely production incident is a tired operator with the wrong alias in scrollback. Torque asks the org to identify itself at the moment of the write, treats anything it cannot verify as production, and refuses a command

that changes the target and writes in the same breath — `sf config set target-org=x && sf data update ...` can re-point the write between the check and the execution, so it is denied as a combination.

3 — It is built to be disbelieved. Torque ships 128 adversarial fixtures and 11 mutators that neuter each guard in turn and require the corresponding attack to succeed — or, for the static scanners, require the check to fail. A guard that cannot be shown to be load-bearing is decoration. Enforcement claims are machine-labelled hook-enforced, harness-enforced, or model-honoured, and the labels are resolved against the live hook set and check registry, so the documentation cannot quietly overstate the code.

GETTING STARTED

03

Setup — clone to working

Prerequisites: **Claude Code** (the hooks are its PreToolUse surface), `git`, `python3` ≥ 3.11 , the Salesforce CLI ≥ 2.60 , and any non-production org — a free Developer Edition is fine. `node` is optional and only affects the live browser check.

STEP 1 — SEE IT WORK BEFORE TRUSTING IT WITH AN ORG

No org, no credentials, no configuration. 24 real cases through the real hooks — 18 attacks it must refuse, 6 ordinary commands it must allow — in about three seconds.

```
git clone https://github.com/omoji-personal/torque && cd torque
python3 bin/torque-demo

Shell indirection — the gate never sees the literal word `sf`.
DENIED x=sf; $x data delete bulk --subject Account --target-org acme-prod
variable-assembled command → Salesforce operation hidden in a shell
assignment value

Path expansion — only becomes the secret's path once bash expands it.
DENIED cat /Users/you/.tor*/secret
glob reaches the signing secret → reference to the trust anchor
(~/.torque) — secret and tokens are operator-only
DENIED p=$HOME/.torque;cat $p/secret
path assembled through a shell variable → reference to the trust anchor

Normal work is untouched. A gate that blocks real work gets switched off.
allowed sf data query --query "SELECT Id FROM Account" --target-org any-org
allowed grep -rn 'sf data delete --target-org' .
allowed echo 'runbook: sf project deploy start' >> notes.md

all 24 behaved correctly
```

STEP 2 — CONNECT AN ORG AND CONFIGURE

You authenticate; the agent never handles credentials. `torque init` refuses to continue if the org classifies as production, creates the trust anchor outside the repo, verifies the hooks are registered, and proves they bind before handing back

control.

```
sf org login web --alias my-dev-org
python3 bin/torque init my-dev-org
```

STEP 3 — PROVE IT AGAINST YOUR OWN ORG

```
npm install          # optional — only the live browser check needs this
python3 harness/validate.py --profile release --target-org my-dev-org
```

Three profiles compose: static (10 checks, **no org needed** — the try-before-you-connect path), capability (24, adds the live cycles), and release (27). `--only <check>` runs one.

STEP 4 — WORK

Open the folder in Claude Code. The hooks fire on every tool call; nothing further is needed. Start read-only — *"audit this org"* — and watch the survey come back. Skills are invoked in plain language: *"add a Priority Score formula field to Account," "set Industry to Technology for all Partner accounts," "log what we just did."*

SCOPE OF THE GATES

Hooks load from the **active workspace**, so they bind when you open this folder. A Claude Code session opened somewhere else loads no gates at all. If the agent will work in other repos, run `python3 bin/torque install-gates` — it registers them at user level with a recorded `TORQUE_HOME`, so they bind everywhere.

WHAT LANDS ON DISK

<code>~/ .torque</code>	Trust anchor, mode 0700, outside the repo — signing secret, approval tokens, session grants, approved Apex bodies
<code>local/audit.log</code>	Every ALLOW / DENY / PROD-WRITE decision, JSON lines, redacted, 0600
<code>local/sessions/<org>.jsonl</code>	Session log and undo data, 0600
<code>local/writable-orgs.json</code>	The write allowlist — a client can read exactly what the agent may touch

All of `local/` is gitignored, and a harness check fails if anything under it is tracked or the ignore rule is removed. Nothing in Torque makes a network call except the `sf` CLI talking to your org.

The operations, worked through

safe-deploy — metadata that is verified, not just accepted

- **Describe first.** Referenced objects and fields are confirmed against the live org. Hallucinated API names are the most common cause of failed deploys; guessing is not permitted.
- **Dry-run** always precedes the real deploy.
- **Deploy.** The gate confirms the target is allowlisted *and* classifies non-production from a live query at that moment.
- **SOQL-verify** the component is present — by querying the org, not by reading the deploy result.
- **FLS-verify.** A formula field deployed without an accompanying PermissionSet is invisible to everyone including System Administrator — a silent failure the deploy status will happily call success.
- **Functional check** (model-honoured): exercise it the way a user will meet it.

mass-update — a bulk change you can walk back

"Set Industry to Technology for all Accounts where Type is Partner." The refusal to act blindly is the feature — current values are queried, the exact Id set and per-field diff built, before-values captured, and only the records that actually differ are written:

```
matched 412 · already correct 118 · will change 294
```

Touching only real changes keeps audit history clean and avoids firing automation on records that were already correct. If the diff is empty the skill stops and issues nothing — a no-op bulk write still stamps `LastModifiedDate` and fires automation on records nobody changed. Undo refuses any record whose current value no longer matches what the run wrote.

The approval round-trip

This is the whole thesis in six lines:

```
you      "Delete the 400 test Leads in my-dev-org."

agent    sf data delete bulk --subject Lead --file ids.csv --target-org my-dev-org
DENIED bulk-delete requires operator-present approval. An agent cannot mint
        an HMAC-signed token. Run: torque approve my-dev-org bulk-delete

you      $ python3 bin/torque approve my-dev-org bulk-delete
        Approve bulk-delete on org 00D...? [type YES]: YES
        approved: single-use token, expires in 5 minutes

agent    (retries — token consumed, delete runs; a second attempt is denied)
```

Operations that need approval: apex, bulk-delete, bulk-write, where-delete, where-update, destructive-metadata, write. Tokens live five minutes and are single-use. For production, add --prod and type WRITE PRODUCTION; for a window, --session <minutes> (≤ 120 , revocable with --end-session).

Anonymous Apex — the strictest path

Inline Apex is refused outright: a body passed on the command line or stdin cannot be bound to an approval. Instead the body lives in a file, torque approve --apex-file prints it *in full* at your terminal — never truncated, because approving code you only half saw is not approval — and writes a read-only copy named for its own SHA-256. The gate re-hashes that file at run time, so an Apex payload edited after approval fails on digest mismatch. Apex routed through MCP is refused entirely and redirected to the file path.

audit-org, session-log, orgs

audit-org runs a fixed read-only battery through the org-explorer subagent, which carries no Edit or Write tools and whose tool list is machine-checked by the harness. **session-log** (torque log) appends a redacted, 0600 entry with before → after values and computes a reversible flag, warning when there is no before-value to reverse to. **orgs** handles connection, live classification, and the write allowlist — connecting is sf org login web; deciding whether Torque may write is an explicit per-org operator decision.

The rules the agent carries

org-safety	Production read-only by default; classification precedence; delete test records by Id only, never by title or date match
live-verification	Never guess an API name; describe before you reference; namespace resolution in managed-package orgs
deployment	Deploy order, formula-field FLS, dry-run discipline, post-deploy verification, the anti-pattern list
platform-quirks	A dated, sourced catalog of platform behaviour that silently produces wrong answers
browser-testing	Frontdoor auth, Lightning timing, shadow DOM, never wait on network-idle
working-discipline	Session logs, privacy posture, QA honesty — name exactly which layers ran
validation	Nothing ships without the harness green against a live org

Each rule is labelled with what actually backs it — a hook, a harness check, or the model's judgment. Only platform-quirks is the last of those, and it says so. The rule files are also byte-capped and the cap is enforced, which is why they read terse: context spent on a rule is context not spent on the work.

The limit — how production is gated

- L1 **Credentials.** Connect production read-only. Torque stores no org credentials — the `sf` CLI is the only credential path. This is the real boundary and the honest answer to "what if everything else fails."
- L2 **Authorization by identity.** A write is allowed only when its target is on an explicit allowlist *and* classifies non-production from a live `Organization` query at that moment. `IsSandbox` must be a real boolean — a malformed "false" string is truthy in Python and would flip production to sandbox. Scratch-org evidence matches on `org Id` only, because alias equality is forgeable. The classification cache is never consulted on the authorization path, because the agent can write to it.
- L3 **Deterministic gates.** Two `PreToolUse` hooks, registered across Bash, MCP, Edit/Write and Read, share one expansion-aware parser that authorizes by *parsing* *argv* and defaults to deny. Every MCP tool call goes through the same gates: an org-touching tool whose verb is unrecognised is treated as a write, and names are decomposed across `_`, `-`, camelCase and acronym boundaries so `bulkDeleteRecords` and `HTTPDelete` both classify.
- L4 **Production override — deliberate, not impossible.** Real work includes deploying to a client's prod, so it is gated on a present operator rather than walled off. `torque approve <org> <op> --prod` mints a single-use token; `--session <min>` opens a bounded, revocable window. Every production write is audited as a `PROD-WRITE` line in `local/audit.log`.
- L5 **Verified enforcement.** Every rule is labelled hook-enforced, harness-enforced or model-honoured, and the labels resolve against the live hook set and check registry. The first version of that checker validated only the hook labels — so a harness-enforced label naming a check that had never existed passed silently. It was caught by the discipline it enforces.

FAILING CLOSED HAS TO SURVIVE THE HOST, NOT JUST THE CODE

A hook that Claude Code kills for running long exits at a status that means *allow*. So bounding each `sf` callout was not enough: classification makes two sequential calls, and two 45-second caps is 90 seconds — past the host's timeout, and therefore a fail-open window. Torque sets one wall-clock budget for the whole gate, clamps every callout to what remains, and treats an exhausted budget as an unverifiable org — which means production, which means deny. A hung `sf` on the `PATH` is a live test case in the harness.

WHY THE AGENT CANNOT SELF-APPROVE

The gate runs *before* bash expands, so it judges what a token *could* become, not what it says. Every argument is converted to an absolute glob pattern; inline `VAR=` assignments are resolved in order; undefined variables are treated as bash treats them; command substitution becomes a wildcard; and matching uses a dynamic-programming glob-prefix matcher, because an earlier positional compare

misaligned on **. Four escalating attacks — `~/ .torq*/secret`, `a=.tor;b=que;cat ~/ $a$b/secret`, a path assembled across three variables, and `/Users/**/.../. [t]orque/sec[r]et` — all die on that one property. Tokens are HMAC-signed with a secret the agent's Bash, Edit *and* Read tools are all denied; the issuer refuses unless attached to a real login TTY clear of the agent; and a token is claimed by an atomic rename *before* it is read, so two concurrent gates cannot both spend it.

A FLOOR NO TOKEN LIFTS

A short list of sObjects is shielded from bulk-delete shapes on **every** org, approved or not — `harness/checks/protected-objects`, seeded with Account, Contact and Opportunity. It is a floor, not an authorization check, and the shield reaches inside an approved Apex body too. Edit it yourself; the agent's tools cannot.

THREAT MODEL — WHERE THE BOUNDARY HONESTLY SITS

The gates stop two categories. **Accidents, with certainty:** a wrong alias, a forgotten `--target-org` that would silently use the default org, a fat-fingered bulk delete. This is the dominant real-world risk and it is closed structurally. **Enumerable circumvention** through the tool surface: indirection, grouping, wrapper runners, `eval` and here-strings, `xargs stdin`, legacy `force:data:*` verbs, glued redirects, `cd-desync` aimed at the gate's own files, forged tokens, and destructive operations routed through MCP under a non-obvious name. Each has a named fixture that must deny.

What no PreToolUse hook can stop is a same-user actor who steps *outside* the tool surface: writing a script and running it, so `sf` executes as a subprocess the hook never sees; forging a login session to fake operator presence; or reading the anchor by OS means the gated tools never touch. Those are the filesystem and credential boundary, not something a hook adjudicates. For them the load-bearing defence is L1 — connect production read-only, and never leave a production org authenticated in an autonomous session. **Even a write that fully bypasses the gate then meets an org that fails the live non-production check with no operator override present.** A PATH shim that closes the subprocess channel is roadmap, not shipped; v1 raises the direct-tool-surface bar as high as a parsing gate honestly can and states the rest plainly.

TROUBLESHOOTING

06

When it refuses

Denials print as `TORQUE GATE DENY [hook] reason`, and every decision — allow and deny alike — lands in `local/audit.log`. Common symptoms:

Nothing is gated	The session isn't in this workspace. Hooks load per-workspace — run <code>torque install-gates</code> for user-level registration.
<code>init</code> refuses my org	Working as designed: it will not configure against an org that classifies production, and an org it cannot classify counts as production. Use a Developer Edition org.
Everything says PRODUCTION	Usually expired <code>sf</code> auth — classification fails, and failure means production. Re-authenticate. Correct fail-safe, confusing symptom.
Denied despite a token	Either the token expired (5 min), the <code><op></code> didn't match the operation, or the <code>sObject</code> is on the protected list — which no token lifts.
<code>clean_ip</code> reports N/A	Expected on any clone but the author's: it needs a private pattern list. An honest N/A, not a failure, and it does not degrade the verdict.
<code>browser_render</code> BLOCKED	No Chromium — <code>npx playwright install chromium</code> . Detection is macOS-path-based today.
A denial you believe is wrong	<code>torque approve <org> --session 30</code> unblocks you for a bounded window. Then please report it — a false denial is a bug worth having; see <code>SECURITY.md</code> .

EVIDENCE

07

Proving it — the validation harness

A LIVE CAPABILITY CYCLE

Deploy a custom field to your org → verify it exists by SOQL → verify field-level security → hard-delete it → confirm zero residue. Then a mass-update with a working undo, and a real headless Lightning render. Nothing is mocked; if the org disagrees, the check fails.

EXIT CODES ARE NOT PROTECTION

An exit code proves a hook returned deny. It does not prove the org was protected. `harness/tests/live_matrix.sh <org>` closes that gap: it gates each operation, *executes* the allowed ones for real, and re-queries the org after every one — data untouched after a denial, the record actually gone after an approved delete, residue zero at teardown. It also puts a hanging stub `sf` on the `PATH` to prove the gate budget denies rather than hangs.

128 ADVERSARIAL FIXTURES

Every attack class found across the audit rounds, each a named, runnable regression test. Just over a fifth assert that *ordinary work is allowed*, because a gate that blocks real work gets switched off.

Each temporarily neuters one guard and requires the corresponding attack to succeed — or, for the static scanners, requires the check to fail. Ten run without an org; the eleventh needs one, because the check it neuters queries the org. Three exercise the clean-IP scan against a private pattern list and report as operator-only on your clone rather than pretending to pass.

```
python3 harness/validate.py --self-test

· clean_ip mutators (3): operator-only — the private pattern list is not
  published, so they are skipped rather than failed
✓ secret_scan mutator: expected FAIL, got FAIL
✓ anchor-guard mutator: expected NOT-deny, got exit 0
✓ destructive token mutator: expected ALLOW, got exit 0
✓ redirect-detection mutator: expected NOT-deny, got exit 0
✓ wrapper (wrapped_sf) mutator: expected NOT-deny, got exit 0
✓ expansion-awareness mutator: expected NOT-deny, got exit 0
✓ glob-matcher (_glob_reaches) mutator: expected NOT-deny, got exit 0
· redaction mutator: needs --target-org (the session-log check queries an
  org) — skipped rather than counted

self-test: all runnable mutators caught — NOT RUN: clean_ip x3
(operator-only); redaction (needs --target-org)
```

That last line is the design. A skip is never allowed to read as a pass — an earlier version printed "ALL MUTATORS CAUGHT" while silently omitting the org-bound mutator, which is exactly the false green this harness exists to prevent. It was found by auditing this guide.

THE HARNESS'S OWN FAILURE MODES

Three, each discovered the hard way and left in the code as comments. Mutators spawn `validate.py` as a subprocess, and the child re-mutated the same file — observed for real as stacked `MUTANT` lines in `lib.py`, now prevented by a recursion guard. The historical-blob mutator plants through git plumbing onto a throwaway ref, because an earlier `--hard` version destroyed uncommitted work. And the redaction mutator patches the live module object rather than the file, because the module is already imported and a file edit would leave the cached function in place and report a **false pass** — a mutation test that would have proven nothing.

The dated run log is `harness/VALIDATION.md`. It is meant to be reproduced, not believed.

PROVENANCE

08

Where it came from

The method is years old; this repository is days old. If you check the git history you will find a short one, and that is the honest shape of it: the method was learned in production across more than twenty-five live Salesforce orgs, and

extracting it into something portable was the fast part. What took the time is
logged per-commit in `harness/VALIDATION.md`.

-
- 01 **The pattern.** Agents were genuinely useful in orgs, and the near-misses were never exotic. Wrong alias. A deploy that reported success and changed nothing a user could see. A bulk update with no way back. Prompt instructions — "be careful, don't touch prod" — do not bind a model under load.
-
- 02 **The insight.** The fix isn't a better prompt, it's a layer that doesn't negotiate. Enforcement has to live outside the model's context, parse what is actually being run, and default to deny — because the model that is confused is the one asking.
-
- 03 **The design, audited before it was built.** Nine rounds of multi-model adversarial review against the plan — independent models arguing to falsify it, every finding required to carry a runnable command that reproduces it — before a line of it existed.
-
- 04 **The build, attacked after.** Then four adversarial rounds against the working code, including a reviewer whose job was to run each exploit against a real `sf` CLI rather than reason about it. Every finding closed with a general fix and a named fixture — parsing `argv` instead of matching text, judging what a glob could expand to, failing closed on any indirection it can't resolve.
-
- 05 **And again on this guide.** Four more lenses — fact-verification, design, a hostile buyer, and completeness — were run against the document you are reading. They found three fabricated behaviours, a command the deny messages told you to run that did not exist, and the false-green in the self-test above. All fixed; that's what the process is for.

Reference

COMMANDS — PYTHON3 BIN/TORQUE <COMMAND>

demo	24 real cases through the real hooks. No org, no credentials, ~3s.
init <org>	Clone → configured. Refuses production; creates the trust anchor; verifies the gates bind.
approve <org> <op>	Mint a single-use token (5 min). --prod requires typing WRITE PRODUCTION; --session <min≤120> opens a window; --end-session revokes; --apex-file for Apex. Operator TTY only.
install-gates	Register the hooks at user level so they bind outside this workspace.
log	Append a redacted session entry with before → after values.
attest	Emit machine-readable attestations of what was verified.
frontdoor	Authenticated Lightning session for browser verification — writes the URL to a 0600 file and prints only the path.
harness/validate.py	--profile static capability release • --self-test • --target-org • --only

LAYOUT

.claude/skills/	The five operations, each a documented sequence
.claude/rules/	The seven rule files, byte-capped, each labelled with what backs it
.claude/agents/	org-explorer (read-only) • hostile-qa (adversarial reviewer)
hooks/	The shared expansion-aware parser and the two gates
harness/	27 checks, 128 fixtures, 11 mutators, live_matrix.sh, the run log
SECURITY.md	Disclosure policy — 72-hour confirmation, 90-day ceiling, fixtures credited

Torque is open source under MIT, and it is meant to be run rather than read. Clone it, point it at a free Developer Edition org, and watch it refuse things. Omid Mojtahedi